

Práctica: Entidades del dominio de solicitudes

Paso a paso — modelar usuarios, roles, solicitudes y su detalle

Curso	Técnico de Desarrollo Full Stack con C# y Angular
Sesión	Sesión 3 — Modelamiento de datos del dominio
Docente	Edwin Barrientos
Duración estimada	60–90 minutos
Nivel	Básico (guía principal de la sesión)
Entorno	.NET 8 SDK · Visual Studio 2022 Community o VS Code · PostgreSQL o SQL Server Express

1. Objetivos

- Modelar las entidades principales del sistema de solicitudes de dinero.
- Expresar relaciones de uno a muchos y de muchos a muchos en C#.
- Usar claves foráneas junto a propiedades de navegación.
- Centralizar los campos de auditoría en una clase base.

2. Desarrollo paso a paso

Paso 1 — La clase base de auditoría

En un flujo de aprobación toda entidad debe registrar quién la creó y quién la modificó. Crea la carpeta Entidades en el proyecto Domain.

Domain/Entidades/EntidadAuditable.cs

```
namespace SolicitudesApp.Domain.Entidades;

public abstract class EntidadAuditable
{
    public DateTime FechaCreacion { get; set; } = DateTime.UtcNow;
    public string CreadoPor { get; set; } = "";
    public DateTime? FechaModificacion { get; set; }
    public string? ModificadoPor { get; set; }
}
```

Paso 2 — Usuario y Rol

Un usuario puede tener varios roles y un rol lo tienen varios usuarios: es una relación de muchos a muchos.

Domain/Entidades/Usuario.cs

```
namespace SolicitudesApp.Domain.Entidades;

public class Usuario : EntidadAuditable
{
    public int Id { get; set; }
    public string Nombre { get; set; } = "";
    public string Correo { get; set; } = "";
    public string PasswordHash { get; set; } = "";
    public bool Activo { get; set; } = true;

    // Muchos a muchos
    public ICollection<Rol> Roles { get; set; } = new List<Rol>();

    // Uno a muchos: las solicitudes que este usuario presento
    public ICollection<Solicitud> Solicitudes { get; set; } = new List<Solicitud>();
}

public class Rol
{
    public int Id { get; set; }
    public string Nombre { get; set; } = ""; // Usuario, Revisor, Aprobador, Tesoreria
    public decimal LimiteAprobacion { get; set; }

    public ICollection<Usuario> Usuarios { get; set; } = new List<Usuario>();
}
```

Nunca guardes contraseñas en texto plano: la entidad almacena PasswordHash. En la sesión 13 se genera ese hash con BCrypt o PBKDF2 al momento del registro.

Paso 3 — El enum de estados

Domain/Entidades/EstadoSolicitud.cs

```
namespace SolicitudesApp.Domain.Entidades;

public enum EstadoSolicitud
{
    Borrador    = 1,
    Enviada     = 2,
    EnRevision  = 3,
    Aprobada    = 4,
    Rechazada   = 5,
    Pagada      = 6
}
```

Paso 4 — Solicitud y su detalle

La solicitud es el documento central y cada línea de gasto es un SolicitudDetalle. Se declara como partial porque en la práctica intermedia se le agregan los métodos de la máquina de estados en otro archivo.

Domain/Entidades/Solicitud.cs

```
namespace SolicitudesApp.Domain.Entidades;

public partial class Solicitud : EntidadAuditable
{
    public int Id { get; set; }
    public string Motivo { get; set; } = "";
    public decimal MontoTotal { get; set; }
    public EstadoSolicitud Estado { get; set; } = EstadoSolicitud.Borrador;
    public DateOnly FechaRequerida { get; set; }

    // Clave foranea + propiedad de navegacion
    public int SolicitanteId { get; set; }
    public Usuario? Solicitante { get; set; }

    // Uno a muchos
    public ICollection<SolicitudDetalle> Detalles { get; set; }
        = new List<SolicitudDetalle>();
    public ICollection<Aprobacion> Aprobaciones { get; set; } = new List<Aprobacion>();

    // Uno a uno
    public MovimientoTesoreria? Movimiento { get; set; }

    // El total se recalcula a partir del detalle
    public void RecalcularTotal() => MontoTotal = Detalles.Sum(d => d.Importe);
}

public class SolicitudDetalle
{
    public int Id { get; set; }
    public int SolicitudId { get; set; }
    public Solicitud? Solicitud { get; set; }

    public string Concepto { get; set; } = "";
    public int Cantidad { get; set; } = 1;
    public decimal PrecioUnitario { get; set; }
    public decimal Importe => Cantidad * PrecioUnitario;
}
```

Clave foránea y navegación juntas: SolicitanteId permite filtrar sin cargar el usuario completo, y Solicitante da acceso al objeto cuando si hace falta. Entity Framework reconoce el patron por convencion.

Paso 5 — Aprobacion y MovimientoTesoreria

Aprobacion guarda el historial del flujo. MovimientoTesoreria cierra el proceso con el pago.

Domain/Entidades/Aprobacion.cs

```
namespace SolicitudesApp.Domain.Entidades;

public class Aprobacion
{
    public int Id { get; set; }

    public int SolicitudId { get; set; }
    public Solicitud? Solicitud { get; set; }

    public int ResponsableId { get; set; }
    public Usuario? Responsable { get; set; }

    public EstadoSolicitud EstadoResultante { get; set; }
    public string? Comentario { get; set; }
    public DateTime Fecha { get; set; } = DateTime.UtcNow;
}

public class MovimientoTesoreria
{
    public int Id { get; set; }

    public int SolicitudId { get; set; } // unica: relacion uno a uno
    public Solicitud? Solicitud { get; set; }

    public decimal MontoPagado { get; set; }
    public string Medio { get; set; } = ""; // Transferencia, Cheque, Efectivo
    public string Referencia { get; set; } = "";
    public DateTime FechaPago { get; set; } = DateTime.UtcNow;
}
```

Paso 6 — Probar el modelo en memoria

Sin base de datos todavía: solo verifica que las relaciones se pueden armar y que el total se calcula bien.

Program.cs

```
var solicitante = new Usuario { Id = 1, Nombre = "Ana Torres", Correo = "ana@empresa.com" };

var solicitud = new Solicitud
{
    Id = 1,
    Motivo = "Equipamiento del area de TI",
    SolicitanteId = solicitante.Id,
    Solicitante = solicitante,
    FechaRequerida = DateOnly.FromDateTime(DateTime.Today.AddDays(15)),
    CreadoPor = solicitante.Correo
};

solicitud.Detalles.Add(new SolicitudDetalle {
    Concepto = "Laptop", Cantidad = 3, PrecioUnitario = 3200m });
solicitud.Detalles.Add(new SolicitudDetalle {
    Concepto = "Monitor", Cantidad = 3, PrecioUnitario = 750m });
solicitud.Detalles.Add(new SolicitudDetalle {
    Concepto = "Teclado", Cantidad = 3, PrecioUnitario = 120m });

solicitud.RecalcularTotal();

Console.WriteLine($"Solicitante : {solicitud.Solicitante?.Nombre}");
Console.WriteLine($"Motivo : {solicitud.Motivo}");
Console.WriteLine($"Lineas : {solicitud.Detalles.Count}");
Console.WriteLine($"Monto total : {solicitud.MontoTotal:N2}");
Console.WriteLine($"Estado : {solicitud.Estado}");

foreach (var d in solicitud.Detalles)
    Console.WriteLine($" {d.Concepto,-10} {d.Cantidad} x " +
        $"{d.PrecioUnitario:N2} = {d.Importe:N2}");
```

3. Retos adicionales

1. Agrega la entidad CentroCosto y relacionala con Solicitud como uno a muchos.
2. Agrega la propiedad calculada DiasEnProceso a Solicitud usando FechaCreacion.
3. Dibuja el diagrama entidad relación del modelo completo en una herramienta de tu preferencia.
4. Justifica por escrito por qué MontoTotal se guarda además de poder calcularse.

4. Entregables

- El proyecto Domain contiene las seis entidades y el enum, y compila sin errores.
- El programa de prueba muestra la solicitud con su detalle y el total correcto.
- Cada relación usa clave foránea y propiedad de navegación.
- Las entidades que lo requieren heredan de EntidadAuditable.

Técnico de Desarrollo Full Stack con C# y Angular · Sesión 3 · Docente: Edwin Barrientos · New Horizons